



Taller — Máquina IAS / Von Neumann

ICCD332 Arquitectura de Computadores

Almeida Anthonny, Sagñay Micaela, Molina Jhanavi y Haro Damian

2026-06-12

1. Objetivos

- Comprender el ciclo de captación, decodificación y ejecución de la máquina IAS (arquitectura Von Neumann).
- Identificar y aplicar el conjunto de instrucciones IAS para traducir código máquina a lenguaje ensamblador y a notación RTL.
- Simular mentalmente la ejecución de un programa en la máquina IAS para determinar el estado final de los registros.

2. Instrucciones

Complete las solicitudes del taller y suba el archivo `.org` y `.pdf` al aula virtual. Use las combinaciones de teclado para trabajar en Emacs.

3. Recursos

Para este taller consulte el libro *Organización y arquitectura de computadores* disponible en EPN-SHAREPOINT (Clic aquí) y use su WSL con Emacs para editar el archivo.

3.1. Observaciones y Recomendaciones

- Tenga en cuenta la ruta a las imágenes y a la bibliografía. Se le recomienda que realice una clonación o un fork del repositorio de la clase a fin de que no tenga problemas generando el documento.
- Para la generación correcta de archivos la configuración básica debe estar operativa y funcional (es decir, Emacs, L^AT_EX).

4. Taller: Máquina de Von Neumann

La máquina de Von Neumann realiza operaciones en un ciclo repetitivo de **captación**, **decodificación** y **ejecución**. Para esto, el computador IAS usa una memoria de 40 bits. Cuando la memoria se usa numéricamente, el dato representa un número en complemento a 2; donde el bit inicial corresponde al signo. Cuando la memoria se usa para el registro de instrucciones, se divide en dos partes de 20 bits cada una. Los 8 primeros bits de cada parte corresponden al *opcode* y los restantes 12 bits de cada parte a la *dirección (operando)* desde la que se debe leer o a la que se debe escribir.

4.1. Conjunto de Instrucciones IAS

Complete la Tabla 1 de instrucciones del computador IAS [1, p.47] usando la notación RTL¹. Se suministra la Tabla en código L^AT_EX para tal efecto.

Para editar la Tabla 1 ubíquese en el bloque `#+begin_export latex` y ejecute el comando `C-c`'. Esto dividirá el buffer en dos ventanas. El cursor salta a la nueva ventana y le permite editar la tabla. Dentro de la nueva tabla el modo predominante cambiará automáticamente a L^AT_EX, lo que le permite hacer cambios con la asistencia del IDE. Para salir y aceptar los cambios ejecute de nuevo `C-c`'.

- M equivale a *Alt*
- x es la tecla *x*
- RET es presionar *Enter*

Una vez activado el modo L^AT_EX el siguiente código recibe colores sobre las palabras clave.

Cuadro 1: Instrucciones Máquina IAS

Opcode	Opcode Hex	Símbolo	RTL
<i>Transferencia de datos</i>			
00001010	0x0A	LOAD MQ	$[AC] \leftarrow [MQ]$
00001001	0x09	LOAD MQ, M(X)	$[MQ] \leftarrow [X]$
00100001	0x21	STOR M(X)	$[X] \leftarrow [AC]$
00000001	0x01	LOAD M(X)	$[AC] \leftarrow [X]$
00000010	0x02	LOAD -M(X)	$[AC] \leftarrow -[X]$
00000011	0x03	LOAD M(X)	$[AC] \leftarrow [X] $
00000100	0x04	LOAD - M(X)	$[AC] \leftarrow - [X] $
<i>Salto incondicional</i>			
00001101	0x0D	JUMP M(X,0:19)	$[PC] \leftarrow M(X) \text{ izq.}$
00001110	0x0E	JUMP M(X,20:39)	$[PC] \leftarrow M(X) \text{ der.}$
<i>Salto condicional</i>			
00001111	0x0F	JUMP+ M(X,0:19)	si $[AC] \geq 0$: $[PC] \leftarrow M(X) \text{ izq.}$
00010000	0x10	JUMP+ M(X,20:39)	si $[AC] \geq 0$: $[PC] \leftarrow M(X) \text{ der.}$
<i>Aritmética</i>			
00000101	0x05	ADD M(X)	$[AC] \leftarrow [AC] + [X]$
00000111	0x07	ADD M(X)	$[AC] \leftarrow [AC] + [X] $
00000110	0x06	SUB M(X)	$[AC] \leftarrow [AC] - [X]$
00001000	0x08	SUB M(X)	$[AC] \leftarrow [AC] - [X] $
00001011	0x0B	MUL M(X)	$[AC, MQ] \leftarrow [MQ] \times [X]$
00001100	0x0C	DIV M(X)	$[MQ] \leftarrow [AC]/[X]$; $[AC] \leftarrow \text{resto}$
00010100	0x14	LSH	$[AC] \leftarrow 2 \times [AC]$
00010101	0x15	RSH	$[AC] \leftarrow [AC]/2$
<i>Modificación de direcciones</i>			
00010010	0x12	STOR M(X,8:19)	$M(X)_{8:19} \leftarrow [AC]_{28:39}$
00010011	0x13	STOR M(X,28:39)	$M(X)_{28:39} \leftarrow [AC]_{28:39}$

¹Register Transfer Language o Lenguaje de Transferencia de Registros.

4.2. Ejercicio

En la máquina IAS, las instrucciones se dividen en dos segmentos: i) izquierdo, desde el bit 0 a 19, y ii) derecho, desde el bit 20 al 39. Primero se ejecuta el lado izquierdo y luego el derecho. El contador de programa inicia en la posición $[PC] \leftarrow 300$. El conjunto de instrucciones del computador o ISA² está definido en la Tabla 1. A fin de terminar el programa, se agrega la instrucción **0x99** que lleva al computador al estado de HALT. Conteste las siguientes preguntas considerando la Tabla 2 como el mapa de memoria de la máquina y obtenga el programa en lenguaje ensamblador y en RTL.

Cuadro 2: Mapa de Memoria				
Dirección	$Opcode_1$	X_1	$Opcode_2$	X_2
0x300	01	940	06	941
0x301	21	940	14	000
0x302	21	941	02	940
0x303	21	940	99	000
...	...	...	...	...
0x940	00	000	00	005
0x941	00	000	00	002

4.2.1. Preguntas

1. ¿Qué resultado se tiene al final en el registro del acumulador?

Respuesta: $[AC] \leftarrow 0000000003$

2. ¿Se sobrescribe algún registro como resultado de la ejecución? Si es verdadero, indique qué registro y con qué valor. Respuesta:

- $[940] \leftarrow 0000000003$
- $[941] \leftarrow 0000000006$
- $[AC] \leftarrow 0000000003$

3. ¿Cuál es el valor del bit de Carry? ¿Qué operación modifica el bit? Respuesta:

- $Carry = 1$
- Lo genera la instrucción **SUB M(941)** en la dirección **0x300 der**. La resta se ejecuta internamente como suma en complemento a 2: $0101 + 1110 = 10011$, cuyo carry-out del bit más significativo es **1**. Aunque LSH también desplaza bits, la bandera de carry aritmético queda fijada por esta operación de sustracción.

4.2.2. Código RTL

Escriba en notación de transferencia de registros el programa que ejecuta el computador IAS.

- $[AC] \leftarrow [940]$
- $[AC] \leftarrow [AC] - [941]$
- $[940] \leftarrow [AC]$
- $[AC] \leftarrow 2 \times [AC]$
- $[941] \leftarrow [AC]$

²Instruction Set Architecture: conjunto de instrucciones que la máquina IAS puede interpretar y ejecutar.

- $[AC] \leftarrow -[940]$
- $[940] \leftarrow [AC]$
- HALT

4.2.3. Código Assembler

Escriba el código Assembler del programa que ejecuta el computador IAS.

```

LOAD M(940)      ; 0x300 izq -> [AC] = 0000000005
SUB M(941)        ; 0x300 der -> [AC] = 0000000005 - 0000000002 = 0000000003 (Carry = 1)
STOR M(940)       ; 0x301 izq -> [940] = 0000000003
LSH              ; 0x301 der -> [AC] = 0000000003 * 2 = 0000000006
STOR M(941)       ; 0x302 izq -> [941] = 0000000006
LOAD -M(940)      ; 0x302 der -> [AC] = -[940] = -0000000003 = FFFFFFFFD
STOR M(940)       ; 0x303 izq -> [940] = FFFFFFFFD
HALT             ; 0x303 der -> fin del programa

```

5. Equipo de Trabajo

Complete la información de los integrantes del grupo.

Nombre	Email	Rol
Anthonny Almeida	anthonny.almeida@epn.edu.ec	Líder
Micaela Sagnay	micaela.sagnay@epn.edu.ec	Colaborador
Jhanavi Molina	jhanavi.molina@epn.edu.ec	Colaborador
Damian Haro	damian.haro@epn.edu.ec	Colaborador

6. Verificación de Entregables [100 %]

Ejecute C-c C-c sobre los ítems de tarea según se hayan cumplido o no. Si un ítem no pudo realizarse, anote en la sección de problemas las razones.

- ☒ Tabla de instrucciones IAS completada.
- ☒ Preguntas del ejercicio respondidas.
- ☒ Código RTL completo.
- ☒ Código Assembler completo.
- ☒ Revisión de ortografía con `ispell-buffer`.
- ☒ Exportación a PDF con `M-x org-latex-export-to-pdf`.
- ☒ Subir al aula virtual archivo `.org` y `.pdf`.

6.1. Problemas con la tarea / logísticos

- Tarea X_1 no pudo completarse debido a:
- Tarea X_2 no pudo completarse debido a:

Referencias

- [1] William Stallings, Antonio Cañas Vargas y Alberto Prieto Espinosa. *Organización y arquitectura de computadores*. Pearson Educación, 2006.